

Design and FPGA Implementation of an Extended RV32IM Processor with a Custom Mathematical Acceleration Unit

Team Members: Komirisetty Prabhas | Karyampudi Komal | Kasireddi Sai Chandra Kiran Naidu

Roll Numbers: 240101045 | 240101041 | 240101042

Group Number: 19

Course: CS 224 – Hardware Lab

1. System Description

Extends a 3-stage pipelined RV32I RISC-V processor with 8 RV32M instructions (MUL, MULH, MULHU, MULHSU, DIV, DIVU, REM, REMU) and a Mathematical Acceleration Unit (MAU) providing 5 custom instructions: ABS, MAX, MIN, SQRT, LOG2. MAU instructions use the RISC-V custom-0 opcode (0001011). The total instruction set expands from ~40 to 53 instructions. Multi-cycle ops (MUL: 3c, DIV: 32c, SQRT: 16c) use pipeline stall logic; single-cycle ops (ABS, MAX, MIN, LOG2) are combinational.

2. Use Cases

2.1 DSP Filtering

16-tap FIR filter kernel using MUL for tap computation and ABS for magnitude envelope detection. Reduces cycle count vs. software multiply emulation.

2.2 Euclidean Distance Computation

Computes $\sqrt{(x1-x2)^2 + (y1-y2)^2}$ using SUB, MUL, ADD, and SQRT instructions. Used in obstacle avoidance and nearest-neighbour search.

2.3 Data Normalization and Clamping

Clamps sensor data to valid ranges using MAX/MIN, computes absolute errors with ABS, and performs logarithmic scaling with LOG2 for dB conversion.

3. FPGA Relevance

DSP48 Slices: Dedicated 25x18 multipliers on Xilinx FPGAs directly implement our 32-bit multiplier with minimal LUT overhead and high clock frequency.

Fast Carry Chains: CARRY4 primitives enable efficient subtract-and-shift in the non-restoring SQRT algorithm and restoring divider, reducing per-cycle propagation delay.

CLZ as Priority Encoder: The LOG2 instruction maps to a hierarchical LUT-based priority encoder tree, achieving O(1) combinational latency vs. 32-cycle software loop.

Real-World Demo: LEDs, seven-segment displays, UART, and switches provide tangible Input-Processing-Output demonstration, far beyond waveform-only verification.

4. System Scope

4.1 Minimum System (Commitment)

MUL/MULH/MULHU/MULHSU: 32x32 multiplier with correct 64-bit results for all signed/unsigned combinations.

DIV/DIVU/REM/REMU: 32-bit divider with correct edge cases (divide-by-zero, INT_MIN/-1 overflow).

ABS/MAX/MIN: Single-cycle combinational units integrated into pipeline execute stage.

Pipeline integration: Modified execute.v, opcode.vh, IF_ID.v with stall logic for MUL/DIV.

Verification: Module-level testbenches (100+ vectors per instruction) + 3 C programs on FPGA.

4.2 Goal System

SQRT: Non-restoring integer square root, 16-cycle FSM, handles $SQRT(0)=0$ to $SQRT(2^{32}-1)=65535$.

LOG2: CLZ-based $O(1)$ combinational unit using 5-level priority encoder tree. $LOG2(0)$ returns 0.

Full MAU integration: All 5 custom instructions multiplexed through shared control decoder.

5+ C programs on FPGA: FIR filter, Euclidean distance, data clamping, GCD, logarithmic scaling.

4.3 Stretch Goal

UART serial output + seven-segment display driver for real-time result visualization.

Interactive demo mode via FPGA switches to select which program to run.

Vivado synthesis report: LUT, FF, DSP, BRAM, Fmax analysis with comparative benchmarking.

5. Design Goals

Metric	Target
MUL latency	3 cycles (pipelined internal stages)
DIV latency	32 cycles (restoring algorithm, 1 bit/cycle)
SQRT latency	16 cycles (non-restoring, 2 bits/cycle)
ABS/MAX/MIN/LOG2 latency	1 cycle (combinational)
Pipeline stalls	Zero stalls for single-cycle ops; stall asserted for MUL/DIV/SQRT
Correctness	100% pass on 100+ test vectors per instruction; edge cases verified
Resource target	DSP: 2-3 slices; BRAM: 0 extra; LUTs: within FPGA budget
FPGA programs	Minimum 5 C programs compiled with -march=rv32im

6. Implementation Approach

New modules: multiplier.v (MUL/MULH/MULHU/MULHSU), divider.v (DIV/DIVU/REM/REMU), mau_simple.v (ABS/MAX/MIN), sqrt_unit.v (SQRT, 16-cycle FSM), clz_unit.v (LOG2, $O(1)$ tree).

Datapath changes: Extended execute.v with result mux for all new units; updated opcode.vh with RV32M funct7 decode and custom-0 opcode decode; added stall signals from multi-cycle units to IF/ID pipeline register.

Verification: (1) Module-level testbenches with Python-generated golden I/O (100+ vectors each, edge cases covered). (2) Processor-level simulation using extended pipeline testbench. (3) C programs compiled with `-march=rv32im` tested on FPGA with observable outputs.

7. Demonstration Plan

Input: Pre-loaded C programs in instruction memory; operands in data memory or via FPGA switches (stretch goal).

Processing: Processor executes compiled C workloads (FIR filter, Euclidean distance, data clamping, GCD, log scaling) using the extended RV32IM+MAU instruction set.

Output: Results displayed on LEDs (binary), seven-segment displays (decimal), and UART serial output (detailed result dump). Each demo shows clear Input-Processing-Output flow: known inputs, hardware computation, verifiable outputs compared against pre-computed golden reference values.

8. High Level

Timeline

Week	Milestone	Deliverable
23-29 Mar	Project abstract submission; set up Git repo; finalize instruction encodings	PDF submitted; repo initialized with lab baseline code
30 Mar-5 Apr	Block diagram report; implement multiplier.v + tb_multiplier.v (MUL verified end-to-end)	Block diagram PDF; multiplier code + testbench with passing tests
6-12 Apr	Implement divider.v + tb_divider.v; integrate MUL + DIV into pipeline (execute.v, opcode.vh); MAU simple ops (ABS/MAX/MIN) + testbench	Divider + testbench; pipeline with MUL+DIV stalls working in simulation; 1 C program on FPGA
13-19 Apr	Implement SQRT unit + CLZ/LOG2 unit; full MAU integration; pipeline stall for SQRT; near-final system on FPGA	SQRT + LOG2 code + testbenches; all 13 instructions in pipeline; 3+ C programs on FPGA
20-26 Apr	Final integration; 5+ C programs on FPGA; demo video recording; final report; address remaining bugs	Working FPGA demo with all programs; 4-min video; final report PDF

9. Team Work Plan

Member	Primary Responsibilities	Secondary Tasks
Komirisetty Prabhas	Multiplier (multiplier.v): MUL, MULH, MULHU, MULHSU. CLZ unit (clz_unit.v). LOG2 instruction. Pipeline mods (execute.v, opcode.vh).	System integration testing; synthesis/timing analysis.
Karyampudi Komal	Divider (divider.v): DIV, DIVU, REM, REMU. Pipeline stall logic. IF_ID.v and wb.v modifications.	C test program development; golden vector generation (Python scripts).

Kasireddi Sai Chandra Kiran Naidu	MAU simple ops (mau_simple.v): ABS, MAX, MIN. SQRT unit (sqrt_unit.v). MAU top-level integration.	FPGA constraints; top-level wiring; UART/display drivers; demo preparation;
---	--	--

Collaboration: Two team meetings per week. Private Git repo with feature branches and mandatory code review before merge. Minimum 20 commits per member. Shared testbench framework with Python golden-vector generator used by all members.

10. GitHub Link

<https://github.com/itsmeprabhas/CS224>